Module 1: DevOps Fundamentals and Linux Environment

1. **DevOps Principles & SDLC:** Explain the core principles of DevOps. How does the DevOps lifecycle improve upon traditional Software Development Life Cycle (SDLC) models?
2. **Virtualization vs. Containers:** Compare Virtual Machines (e.g., VMware/VirtualBox) with bare-metal systems in terms of architecture, resource utilization, and isolation.
3. **Linux File System & CLI:** Describe the Linux Directory Structure (FHS). Briefly explain the purpose of /etc, /var, /home, /usr, and /root directories.
4. **Permissions & Ownership:** Differentiate between file permissions and file ownership in Linux. Explain how chmod, chown, and chgrp commands are used with a suitable example.
5. **Shell Redirection & Piping:** Explain the concept of standard input (stdin), standard output (stdout), and standard error (stderr) in Linux. How do pipes (|) and redirection operators (>, >>) work together to chain commands?
6. **Package Management & SSH:** Differentiate between Linux package managers (e.g., apt vs yum/dnf). Explain the role of SSH keys and environment variables in automating remote Linux server connections.

Module 2: Version Control and Build Automation

1. **Git Fundamentals:** Differentiate between Centralized Version Control Systems (CVCS) and Distributed Version Control Systems (DVCS). Explain the three states of a file in Git (Working Directory, Staging Area, Repository).
2. **Git Branching Strategies:** Explain the git commands git branch, git checkout, git merge, and git rebase. Under what scenario would you prefer rebase over merge?
3. **Merge Conflicts & Stashing:** What causes a Git merge conflict, and what step-by-step process is followed to resolve it? Explain how git stash is used during urgent hotfixes.
4. **Build Automation:** Define build automation. Explain the typical build stages (compilation, testing, packaging/artifact creation) for Java or JavaScript applications.

5. **Artifact Publishing:** What are build artifacts (e.g., JAR, WAR, npm packages)? Explain why publishing build artifacts to a repository is essential in a DevOps pipeline.
6. **Build Tools & Docker:** How are modern build tools integrated with Docker to create reproducible build environments? Explain the benefits of multi-stage Docker builds.

Module 3: Cloud, Artifact Management and Containerization

1. **Cloud Compute (IaaS):** Explain the Infrastructure as a Service (IaaS) cloud delivery model. List the step-by-step procedure to provision and configure a secure Linux virtual server on a cloud provider (e.g., Linode/AWS).
2. **Artifact Management with Nexus:** What is Sonatype Nexus, and why is an artifact repository needed in DevOps? Describe Hosted, Proxy, and Group repository types in Nexus.
3. **Nexus Maintenance & APIs:** Explain the concept of Blob Stores and Cleanup Policies in Nexus Repository Manager. How can REST APIs be utilized to manage Nexus artifacts programmatically?
4. **Docker Architecture:** Explain the client-server architecture of Docker. Describe the roles of the Docker Client, Docker Daemon, Docker Images, Docker Containers, and Docker Registry.
5. **Dockerfile Instructions:** Write a sample Dockerfile for a simple web application. Explain the purpose of key directives: FROM, WORKDIR, COPY, RUN, EXPOSE, and CMD.
6. **Docker Storage & Networking:** Differentiate between Docker Volumes and Bind Mounts. Briefly explain the default network drivers in Docker (bridge, host, none).
7. **Docker Compose:** What is Docker Compose, and what problem does it solve? Explain the key sections of a docker-compose.yml file used to run a multi-container application (e.g., App + Database).

Based on the attached **DevOps syllabus**, Modules 1–3 cover DevOps/Linux, Git/build automation, and cloud/artifact management/containerization respectively.

Below is a set of **scenario-based questions** suitable for assignments, internal assessment, practical examinations, or question-bank preparation.

Scenario-Based Questions – DevOps Modules 1, 2 & 3

Module 1: DevOps Fundamentals and Linux Environment

Q1. DevOps Lifecycle

A software company develops an e-commerce application. The development team frequently completes new features, but deployment to production takes several days because testing and operations activities are performed manually.

Questions:

1. Identify the problems in the existing software delivery process.
2. Explain how the DevOps lifecycle can address these problems.
3. Propose suitable DevOps practices for improving collaboration between development and operations teams.
4. Explain how continuous integration and continuous delivery would improve the process.

Q2. Virtual Machines

A college wants to provide Linux environments to 50 students for practicing DevOps tools. Purchasing separate physical servers for every student is expensive.

Questions:

1. Explain how virtualization can solve this problem.
2. Differentiate between a physical machine and a virtual machine.
3. Suggest how VMware or VirtualBox can be used in this scenario.
4. What are the advantages and limitations of using VMs for DevOps practice?

Q3. Linux File System

A developer has deployed an application on a Linux server. The application files are stored under /opt, configuration files under /etc, logs under /var/log, and user files under /home.

Questions:

1. Explain the purpose of these Linux directories.
2. Which directory would you use for application logs?

3. How would you locate a particular configuration file from the command line?
4. Write suitable Linux commands to navigate and inspect these directories.

Q4. Linux Commands, Pipes and Redirects

A system administrator wants to find all .log files containing the word ERROR and save the results into a file called error_report.txt.

Questions:

1. Write a Linux command to identify the required log files.
2. Demonstrate the use of pipes in solving this problem.
3. Explain the difference between >, >>, <, and |.
4. How would you count the number of error entries?

Q5. Linux Users, Groups and Permissions

A development server has three users: developer1, developer2, and tester. Application source code should be accessible to developers, while testers should have read-only access.

Questions:

1. How would you create appropriate users and groups?
2. How would you assign ownership of the application directory?
3. Set suitable Linux permissions for this requirement.
4. Explain the meaning of permissions such as rwxr-x---.

Q6. Environment Variables

A Java application requires the variables JAVA_HOME and PATH to be configured before execution.

Questions:

1. What are environment variables?
2. How would you check the current value of JAVA_HOME?
3. How would you temporarily set JAVA_HOME?

4. Why is configuration through environment variables useful in DevOps environments?

Q7. SSH-Based Server Administration

A DevOps engineer needs to remotely access a Linux cloud server from a workstation.

Questions:

1. Explain how SSH can be used for remote administration.
2. Write the general SSH command for connecting to the server.
3. Explain the role of username, IP address and authentication.
4. Why is SSH preferred over insecure remote-login mechanisms?

Module 2: Version Control and Build Automation

The syllabus specifically includes Git repositories and operations such as **commit, branch, merge, rebase, stash, conflict resolution and history management**, along with build tools, artifact creation/publishing and Docker integration.

Q8. Git Repository Management

A development team of five programmers is working on a Java application. Each developer currently keeps a separate copy of the project, causing different versions of the source code to exist.

Questions:

1. Explain how Git can solve this problem.
2. Explain the difference between a local and remote repository.
3. Describe the steps to create a Git repository and push code to a remote repository.
4. Why is version control important in DevOps?

Q9. Git Branching

A team is developing an online shopping application. The main branch contains the stable version, while developers need to independently develop payment and product-search features.

Questions:

1. Design a suitable Git branching strategy.
2. Explain how developers can create feature branches.
3. How can the completed features be merged into the main branch?
4. What problems can occur when multiple developers modify the same files?

Q10. Git Merge Conflict

Two developers modify the same lines in index.html. Developer A pushes the changes first. When Developer B tries to merge their branch, Git reports a conflict.

Questions:

1. What causes a Git merge conflict?
2. Explain the steps to resolve the conflict.
3. Which commands can be used to identify the conflicted files?
4. How should the developer commit the conflict resolution?

Q11. Git Rebase vs Merge

A project manager notices that the Git history contains many unnecessary merge commits and wants a cleaner project history.

Questions:

1. Explain Git merge and Git rebase.
2. Which approach would you recommend for maintaining a clean feature-branch history?
3. What risks are associated with rebasing shared/public branches?
4. Illustrate the workflow using appropriate Git commands.

Q12. Git Stash

A developer is working on a new feature but receives an urgent request to fix a production bug. The current feature is incomplete and should not be committed yet.

Questions:

1. How can Git Stash help in this situation?
2. Write commands to temporarily store the changes.
3. How can the developer restore the changes later?
4. What is the difference between git stash and git commit?

Q13. Build Automation

A Java development team manually compiles source files, downloads dependencies and creates JAR files before every release. This process frequently results in errors.

Questions:

1. Explain how a build automation tool can solve the problem.
2. Describe the typical build lifecycle of a Java application.
3. Explain how dependencies can be managed automatically.
4. Why is automated build generation important in DevOps?

Q14. Artifact Creation and Publishing

A development team successfully builds a Java application. The generated JAR file needs to be shared with the testing and deployment teams.

Questions:

1. What is an artifact?
2. Why should artifacts be stored in an artifact repository?
3. Describe a suitable process for building, versioning and publishing the artifact.
4. What problems may occur if artifacts are shared manually?

Q15. Build Tool and Docker Integration

A company wants to package a Java application into a Docker image after every successful build.

Questions:

1. Explain the relationship between the build tool and Docker.
2. What files/artifacts should be generated before creating the Docker image?

3. Describe the sequence: **Source Code → Build → Artifact → Docker Image**.
4. What advantages does this integration provide?

Module 3: Cloud, Artifact Management and Containerization

Module 3 covers **IaaS/cloud deployment, Linux configuration on cloud servers, Nexus artifact repositories, repository types, REST API, Blob Store, cleanup policies, Docker architecture and commands, volumes, networking, Dockerfile, Docker Compose, and private Docker repositories/deployment**.

Q16. Cloud-Based Application Deployment

A startup has developed a web application and wants to deploy it without purchasing physical servers. The application is expected to receive varying traffic throughout the year.

Questions:

1. Explain how IaaS can be used for this requirement.
2. What resources would need to be provisioned on the cloud?
3. Explain the steps for setting up a Linux server on a cloud platform.
4. What advantages does cloud deployment provide over maintaining physical infrastructure?

Q17. Linux Configuration on Cloud

A company creates a Linux cloud server for hosting a web application. A developer needs remote access, a dedicated user account and appropriate permissions.

Questions:

1. How would you create a dedicated Linux user?
2. How would you configure SSH access?
3. How would you assign appropriate permissions?
4. Why should applications preferably run using dedicated users rather than the root account?

Q18. Nexus Artifact Repository

A software organization generates hundreds of application artifacts during development. Developers currently store JAR files on personal systems and share them through email.

Questions:

1. Identify the problems with this approach.
2. Explain how Nexus can be used as an artifact repository.
3. Explain the purpose of repository types in Nexus.
4. How would centralized artifact management improve the DevOps process?

Q19. Artifact Repository Cleanup

A CI system creates a new artifact every time a developer commits code. After six months, the repository contains thousands of old artifacts and is consuming excessive storage.

Questions:

1. What is a cleanup policy?
2. Why should old artifacts be removed?
3. Design a suitable cleanup strategy.
4. Explain how a Blob Store is related to artifact storage.

Q20. Docker Containerization

A Java application works correctly on the developer's laptop but fails when deployed on another server because of differences in Java versions and dependencies.

Questions:

1. Explain how Docker can solve this problem.
2. Explain the difference between a Docker image and a container.
3. Describe the basic Docker architecture.
4. What advantages does containerization provide for DevOps?

Q21. Dockerfile

A company wants to containerize a Java application. The application JAR file is named app.jar.

Questions:

1. Design a suitable Dockerfile for the application.
2. Explain each major instruction used in the Dockerfile.
3. Write commands to build the Docker image.
4. Write the command to run the container.

Q22. Docker Volumes

A MySQL database is running inside a Docker container. After the container is deleted, all database data is lost.

Questions:

1. Why is the data lost?
2. Explain the purpose of Docker volumes.
3. How would you modify the deployment to persist database data?
4. Why should persistent application data generally not be stored only inside the container's writable layer?

Q23. Docker Networking

A web application consists of two containers: a frontend container and a backend API container. The frontend needs to communicate with the backend.

Questions:

1. Explain how Docker networking can facilitate communication.
2. How can both containers be placed on the same Docker network?
3. How can the frontend access the backend container?
4. What is the advantage of using container names/service names instead of hard-coded IP addresses?

Q24. Docker Compose

A developer needs to run a complete application consisting of:

- Frontend
- Backend API
- MySQL database

Manually creating and configuring three containers is time-consuming.

Questions:

1. Explain how Docker Compose can simplify this deployment.
2. What information should be specified in a Compose file?
3. Explain how networking between the services can be configured.
4. How would you start and stop the complete application?

Q25. Private Docker Repository

An organization develops an internal banking application. Docker images must not be publicly accessible, but multiple deployment servers need to pull the images.

Questions:

1. Why should the organization use a private Docker repository?
2. Explain the process of building and tagging the image.
3. Describe the authentication process required before pushing the image.
4. Explain the workflow:

Developer → Docker Build → Private Repository → Deployment Server → Docker Container

---

Integrated Scenario-Based Questions

These questions combine concepts from **Modules 1, 2 and 3** and are particularly suitable for higher-level examination questions.

Q26. Complete DevOps Workflow

A company develops a Java-based online examination application. Developers use Git, the application is built using a build automation tool, artifacts are stored in an artifact repository, and the final application is deployed as a Docker container on a cloud Linux server.

Tasks:

1. Design the complete workflow from source code to deployment.
2. Explain where Git, the build tool, artifact repository and Docker are used.
3. Explain how Linux and SSH are used during deployment.
4. Identify possible failure points in the workflow and suggest solutions.

Q27. Production Deployment Problem

A development team reports the following problems:

- Developers have different versions of source code.
- Builds are performed manually.
- JAR files are shared through email.
- Applications work differently on different servers.
- Database data is lost whenever a container is deleted.

Questions:

1. Map each problem to an appropriate DevOps solution.
2. Explain how Git addresses the source-code problem.
3. Explain how build automation and Nexus address the build/artifact problems.
4. Explain how Docker addresses environment differences.
5. Explain how Docker volumes address the database problem.

Q28. Design a Containerized Application

An educational institution wants to deploy a student management system consisting of a Java backend and MySQL database. The source code is maintained in Git and the application must be deployed on a cloud Linux server.

Tasks:

1. Design the Git repository structure.
2. Define a build process for the Java application.
3. Explain how the generated artifact can be stored in Nexus.
4. Design the Docker-based deployment.

5. Explain the Docker network and volume requirements.
6. Explain how the application can be deployed securely on the cloud server.

Q29. DevOps Troubleshooting Scenario

A Dockerized application was working correctly yesterday. Today, the deployment fails with the following symptoms:

- Docker image cannot be pulled.
- The deployment server reports authentication failure.
- The application container starts but cannot connect to the database.
- The application log contains errors that need to be investigated.

Questions:

1. Develop a systematic troubleshooting procedure.
2. Which Docker commands would you use?
3. How would you verify repository authentication?
4. How would you troubleshoot Docker networking?
5. How would you inspect container logs?

Q30. End-to-End Case Study

A software company wants to implement the following environment:

Git Repository → Build Automation → Nexus → Docker Image → Private Docker Repository → Cloud Linux Server

The organization wants developers to work independently using Git branches and wants every application release to have a uniquely identifiable version.

Questions:

1. Design the complete architecture.
2. Explain the role of every component.
3. Define an appropriate Git branching and merging strategy.
4. Explain artifact versioning.
5. Explain Docker image tagging.

6. Explain how the image can be securely pushed to and pulled from a private repository.
7. Explain how Linux permissions and SSH should be configured on the cloud server.